Document Summary

This note covers the fundamentals of wheeled mobile robots, including wheel types, geometric configurations, non-holonomic constraints, and key kinematic models like unicycle, differential drive, and car-like robots. It extends to dynamics using Lagrangian methods, path planning techniques such as kinodynamic planning and ProMPs, and contrasts with aerial robots like quadrotors. Practical examples, simulations, and open-source tools are provided for implementation and visualization.

Wheeled Robots Kinematics, Dynamics, and Planning

Introduction to Mobile Robots

Wheeled mobile robots use wheels for locomotion, and their maneuverability depends on several key factors. These include the type of wheels, which refers to their mechanical structure and actuation method, as well as the geometry of the wheels, meaning their relative placement on the chassis. For instance, the Boston Dynamics Handle robot exemplifies advanced wheeled designs that balance stability and agility.

This document focuses on wheeled mobile robots, covering essential topics such as kinematics, non-holonomic constraints, models, dynamics, and planning. To build a strong foundation, we start with the basic components and progress to more complex behaviors.

Types of Wheels

Wheeled robots employ various wheel types, each with distinct degrees of freedom (DoF) and resulting behaviors. Understanding these helps in selecting the right configuration for specific mobility needs. Below, we describe the standard and advanced types.

Fixed Wheel

Fixed Wheel

- *Fixed orientation relative to the chassis.*
- *Can be active (powered) or passive.*
- *Primarily used for straightforward propulsion in a fixed direction.*

> ***Example:*** *In a simple robot cart, fixed wheels provide reliable forward motion but limit turning without differential speeds.*

Steerable Wheel

> ***Steerable Wheel***
>
> - *Variable orientation relative to the chassis, allowing directional changes.*
> - *Typically active to enable precise control over steering.*

> ***Example:*** *The front wheels of a car are steerable, enabling the vehicle to navigate curves by adjusting their angle.*

Castor Wheel

> ***Castor Wheel***
>
> - *Free orientation relative to the chassis.*
> - *Typically passive, meaning it is not powered.*
> - *Automatically aligns with the direction of motion to maintain stability and prevent skidding.*

> ***Example:*** *Office chair wheels are castor types, swiveling freely to follow the push direction without resistance.*

Degrees of Freedom for Standard Wheels

Standard wheels have varying DoF based on their design, which directly affects robot mobility:

- **1 DoF**: Rotation around the axle only, suitable for basic rolling.
- **2 DoF**: Rotation around the axle plus rotation around the ground contact point, allowing some pivoting.
- **3 DoF**: Rotation around the axle, rotation around the ground contact point, and rotation around the caster axis, providing greater flexibility.

Advanced Wheel Types

- **Spherical Wheels**: Offer 3 DoF for rotation in any direction, plus rotation around the ground contact point. These enable smooth omnidirectional movement without changing orientation.
- **Omniwheels (Mecanum wheels, Swedish wheels)**: Provide 3 DoF, including rotation around the wheel axle, rotation around the ground contact point, and rotation around the axle. The small rollers on the wheel sides allow sideways motion, enabling true omnidirectional movement.

> ***Example:*** *Mecanum wheels are used in warehouse robots like those from Amazon, allowing them to strafe between shelves without rotating the entire body.*

Geometry of Wheeled Robots

The arrangement and placement of wheels on the chassis significantly influence a robot's mobility and control capabilities. These geometric configurations form the foundation for developing kinematic models. Configurations vary widely depending on the robot type—for example, a differential drive setup in a small exploration robot versus a car-like geometry in an autonomous vehicle.

To visualize, consider how wheel spacing (e.g., axle length) affects turning radius: closer wheels enable tighter turns but may reduce stability at high speeds.

Non-Holonomic Constraints

In wheeled robotics, non-holonomic constraints limit the instantaneous velocities of the robot but do not restrict the reachable configurations over time. This means the robot cannot move sideways instantly but can reach any position by following allowable paths. These constraints arise from the physical properties of wheels and are crucial for modeling realistic motion.

General Concepts

> ***Kinematic constraint***
>
> *A relation that restricts achievable velocities through dependencies among position coordinates q and their derivatives $\dot{q}$.*

> ***Holonomic***

A constraint that is integrable, resulting in a relation solely in terms of q (e.g., a fixed joint angle in a linkage).

Non-holonomic

A non-integrable constraint that cannot be reduced to a form without $\dot{q}$. It restricts possible trajectories but not the final configurations, and it limits instantaneous motions.

For robotic manipulators, the configuration q determines the starting point; the motion profile is planned on the joints, and integration yields the end-effector pose directly from joint variables. In contrast, for wheeled robots, the configuration is defined by wheel rotation angles ϕ. Given a wheel radius r, the traveled distance is $r\dot{\phi}$, but the same distance can lead to different positions depending on the path taken.

The task space configuration (e.g., robot pose in the world) requires executing the motion, leading to **differential kinematics**, which describe instantaneous velocity relationships. Importantly, closed trajectories in the configuration space (wheel angles returning to start) do not necessarily result in closed paths in the task space (pose returning to start) due to these constraints.

Example: *A robot driving in a circle may end with wheels back to initial angles, but its position has shifted if the circle is not perfectly closed.*

Pure-Rolling Constraints

A standard wheel introduces two key non-holonomic constraints:

- **No slipping**: The velocity of the wheel center must be parallel to the wheel plane. This is non-holonomic and prevents motion normal to the rolling direction.
- **No skidding**: The magnitude of the wheel center velocity must match the wheel's rotational velocity.

These ensure realistic ground interaction without energy loss from sliding.

Unicycle Model

The unicycle model is the simplest representation of wheeled motion: it assumes a single steerable wheel that rolls without slipping and remains stable in an upright position. This model abstracts many real robots and serves as a building block for more complex ones.

Generalized Coordinates

The robot's state is described by:

- Position in the world frame: (x, y).
- Orientation: θ.
- Wheel rotation angle: ϕ.
- Direction n: Orthogonal to the wheel plane.

Pure-Rolling Constraint

The no-slipping condition is expressed as:

> ***No-Slipping Condition***
>
> $$\dot{x} \sin\theta - \dot{y} \cos\theta + r\dot{\phi} = 0$$

In Pfaffian form, this becomes:

$$A(q)\dot{q} = 0$$

where $q = [x, y, \theta, \phi]^T$ is the state vector, and $A(q)$ is the constraint matrix.

Admissible velocities $\dot{q}$ lie in the null space of $A(q)$, meaning they satisfy the constraint violating physics.

No-Slipping Constraint

The specific form of the constraint matrix is:

> ***Constraint Matrix***
>
> $$A(q) = [\sin\theta, -\cos\theta, 0, r]$$

A basis for the space can be found to parameterize allowable motions:

- One basis vector corresponds to pure translation.
- In general, $\dot{q} = N(q)u$, where $N(q)$ spans the null space, and u are control inputs.

Kinematic Model

Under the no-slipping assumption, the model simplifies to:

> ***Unicycle Kinematics***
>
> $$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega$$

Here, v and ω are the inputs: linear velocity and angular velocity, respectively. This equation describes the admissible instantaneous motions of the robot.

Mathematical Example: *If $v = 1$ m/s and $\theta = 45^\circ$ (or $\pi/4$ radians), then $\dot{x} = 1 \cdot \cos(\pi/4) = 0.707$ m/s and $\dot{y} = 1 \cdot \sin(\pi/4) = 0.707$ m/s, showing diagonal forward motion.*

For simulation, a simple Python snippet can integrate this model:

```
import numpy as np
from scipy.integrate import odeint

def unicycle_dynamics(state, t, v, omega):
    x, y, theta = state
    dxdt = v * np.cos(theta)
    dydt = v * np.sin(theta)
    dthetadt = omega
    return [dxdt, dydt, dthetadt]

# Example: Simulate for 10 seconds with constant v=1, omega=0.1
t = np.linspace(0, 10, 100)
initial_state = [0, 0, 0]
solution = odeint(unicycle_dynamics, initial_state, t, args=(1, 0.1))
```

Differential Drive

The differential drive configuration extends the unicycle model using two active fixed wheels on the rear and a front passive castor wheel for support. This setup is common in mobile robots due to its simplicity and effectiveness for navigation.

Description

- The left and right wheels are independently actuated and fixed in orientation.
- A passive castor wheel at the front provides additional support without contributing to propulsion.

Assumptions

To derive the model, we make the following simplifications:

1. Symmetry along the robot's X-axis: The wheels are equidistant from the center (axle length $2L$), with identical radii $R_L = R_R = R$. The mass center lies on the X_R-axis, at a distance c from the origin O_R.
2. Rigid body: All distances (e.g., c) are fixed, with no deformation.
3. Rotation only around the vertical Z-axis: $\dot{\theta}_x = \dot{\theta}_y = 0$, ignoring tilt.
4. The mass center velocity lies in the body frame along X_R.

Pure-Rolling Constraints

These constraints ensure realistic motion:

1. **No skidding**: The lateral velocity at the origin O_R in the robot frame is zero, preventing side slip.
2. **No slipping**: Each wheel's revolution corresponds to traveling its circumference, linking rotational speed to linear speed.

Kinematic Derivation

From the constraints, we obtain:

- $v_L = r\dot{\phi}_L$, $v_R = r\dot{\phi}_R$, where v_L and v_R are the linear speeds of the left and right wheels.
- The overall robot linear velocity: $v = \frac{v_R + v_L}{2}$.
- The angular velocity: $\omega = \frac{v_R - v_L}{2L}$.

Thus, the forward kinematics become:

> ***Differential Drive Kinematics***
>
> $$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega$$
>
> *This is equivalent to the unicycle model, with the effective wheel at O_R.*

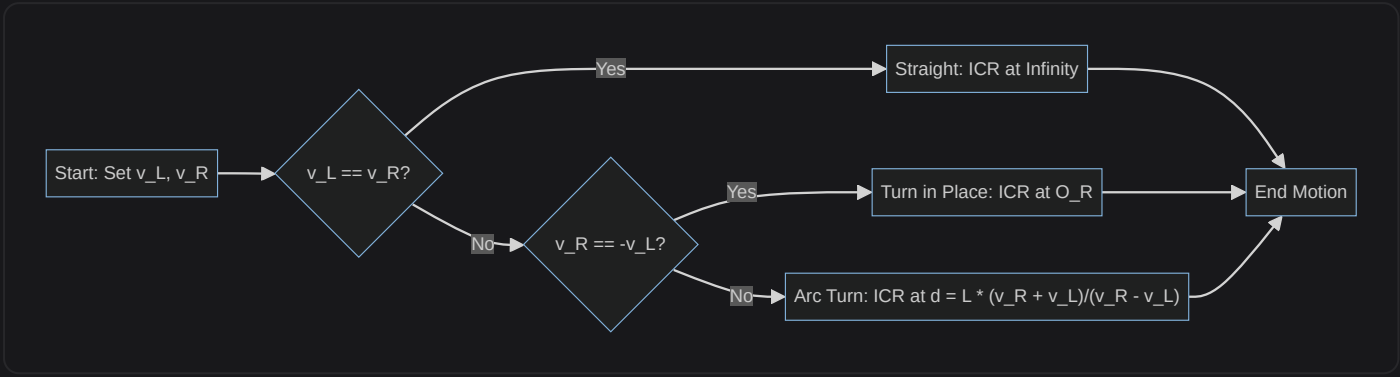
Instantaneous Center of Rotation (ICR)

Under no slipping, wheel velocities are orthogonal to lines from the ICR to the wheel contact points. The ICR determines the turning behavior:

1. If $v_L = v_R$, there is no turn (ICR at infinity, straight motion).
2. If $v_R = -v_L$, the robot turns in place (ICR at O_R).
3. If $v_R = 0$ and $v_L \neq 0$, the robot turns around the right wheel (ICR at distance $d = -L$ from center); similarly for the left.

> ***Example:*** *For $v_L = 1$ m/s, $v_R = 0.5$ m/s, $L = 0.2$ m, then $v = 0.75$ m/s and $\omega = (0.5 - 1)/(2 \times 0.2) = -1.25$ rad/s, indicating a leftward turn.*

To illustrate the ICR positions, consider this simple flowchart of motion modes:



Differential Forward Kinematics

The velocity transformation is:

> ***Forward Kinematics Matrix***
>
> $$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$
>
> *This holds under no slipping and no skidding, mapping body-frame velocities to world-frame changes.*

Inverse Differential Kinematics (in Robot Frame)

To compute wheel speeds from desired v and ω:

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{L}$$

Working in the robot-fixed frame simplifies these relations, as they ignore world orientation.

> ***Example:*** *For desired $v = 1$ m/s and $\omega = 0.5$ rad/s with $L = 0.2$ m, solve for $v_R = v + \omega L = 1.1$ m/s and $v_L = v - \omega L = 0.9$ m/s.*

Car-Like Robots

Car-like robots mimic the kinematics of automobiles, with fixed rear wheels and steerable front wheels. This configuration is ideal for highway driving but introduces more complex steering

dynamics.

Description

- Rear wheels: Fixed orientation, can be active (powered) or passive.
- Front wheels: Steerable, typically active for control.

Assumptions

Similar to differential drive:

1. Symmetry along X_R: Equidistant wheels (axle length $2L$), identical radii $R_L = R_R = R$.
2. Rigid body: Fixed distances, including the rear-to-front axle length l.
3. Rear wheels actuated; front wheels steerable.

Bicycle Model Approximation

To simplify, collapse the paired wheels into single equivalents:

- Rear wheel position: (x, y).
- Front wheel position: (x_F, y_F).
- Front steering angle: ϕ.

Non-Holonomic Constraints

No-slipping applies to both rear and front (bicycle approximation):

1. **Rear no-slipping**:

> ***Rear Constraint***
>
> $$\dot{x}\sin\theta - \dot{y}\cos\theta = 0$$

2. **Front no-slipping**:
   - The front wheel's velocity projection onto its normal direction n_F must be zero.
   - Positions: $x_F = x + l\cos\theta$, $y_F = y + l\sin\theta$.
   - Velocities: $\dot{x}_F = \dot{x} - l\sin\theta\dot{\theta}$, $\dot{y}_F = \dot{y} + l\cos\theta\dot{\theta}$.
   - Constraint: $\dot{x}_F\sin(\theta+\phi) - \dot{y}_F\cos(\theta+\phi) = 0$.

In Pfaffian form:

$$A(q)\dot{q} = 0$$

with $q = [x, y, \theta, \phi]^T$, and

Pfaffian Matrix for Car Model

$$A(q) = \begin{bmatrix} \sin\theta & -\cos\theta & 0 & 0 \\ \sin(\theta+\phi) & -\cos(\theta+\phi) & l\cos\phi & -l \end{bmatrix}$$

Admissible $\dot{q}$ lie in the null space of $A(q)$.

A basis for the null space includes:

- For rear motion: $[\cos\theta, \sin\theta, 0, 0]^T$.
- For steering: A second vector like $[-\sin\theta, \cos\theta, 1, t]^T$, where $t = \frac{1}{\cos\phi}$ ensures orthogonality.

Kinematics Model

The derived model is:

Car-Like Kinematics

$$\dot{x} = u_1\cos\theta, \quad \dot{y} = u_1\sin\theta, \quad \dot{\theta} = \frac{u_1\tan\phi}{l}, \quad \dot{\phi} = u_2$$

- *u_1: Translational input ($u_1 = r\dot{\phi}_R = r\dot{\phi}_L$ for equal rear wheels).*
- *u_2: Steering rate input.*

Mathematical Example: *With $u_1 = 2$ m/s, $\theta = 0$, $\phi = 30^\circ$ ($\pi/6$ rad), $l = 1$ m, then $\dot{\theta} = 2\tan(\pi/6)/1 \approx 1.155$ rad/s, yielding a turning radius of about 0.866 m.*

Kinematics of Center of Mass

The center of mass lies on X_R, at distance b from the rear axle. The velocity expressions are adjusted accordingly using the above kinematics to account for the offset.

Frénet Frame Kinematics

For tasks involving path tracking, the Frénet frame provides a local coordinate system aligned with a reference curve $r(t)$. This frame helps in controlling the robot relative to the path, such as following a road.

Definitions

- Arc length: $s(t) = \int_0^t \|\dot{r}(\tau)\| d\tau$.

- At point s: Tangent $T = \frac{\dot{r}}{\|\dot{r}\|}$, normal $N = \frac{\ddot{r} - (\ddot{r} \cdot T)T}{\|\cdot\|}$, binormal $B = T \times N$.

Frames

- World frame: $F_W = \{O_W, X_W, Y_W\}$.
- Robot frame: $F_m = \{P_m, X_m, Y_m\}$.
- Frénet frame: $F_s = \{P_s, X_s, Y_s\}$, positioned on the path.

Kinematics in Frénet Frame

- Path curvature: $\kappa(s) = \left\| \frac{dT}{ds} \right\|$.
- Lateral offset from P_s to P_m along Y_s: y.
- Robot orientation relative to F_s: ψ.

The kinematics derive the relative velocities between frames, enabling precise tracking control. For instance, the along-path speed $\dot{s}$, lateral speed $\dot{y}$, and heading error $\dot{\psi}$ form the state for a controller.

> ***Example:*** *On a curved path with $\kappa = 0.1\ m^{-1}$, a robot at $y = 0.2\ m$ offset and $\psi = 5^\circ$ can use feedback to adjust u_1 and u_2 for convergence.*

Integrating Differential Kinematics

Dead Reckoning

Unlike manipulators, there is no direct mapping from configuration q to task space pose x. Instead, the pose is estimated by integrating the differential kinematics equations from measured wheel speeds.

- For example, integrate $\dot{x} = v \cos\theta$, $\dot{y} = v \sin\theta$, $\dot{\theta} = \omega$ using encoder data over time.

This process, known as dead reckoning, accumulates position estimates but is prone to errors.

Odometry and Uncertainty

Odometry—the computation of pose from wheel velocities—accumulates uncertainty from several sources:

- Measurement noise in encoders.
- Integration errors over time (drift).
- Slippage due to terrain or acceleration.
- Inaccuracies in calibration (e.g., wheel radius or axle length).

To mitigate, intermittent pose fixes (e.g., from landmarks or GPS) can reset the integration and prevent unbounded drift. Techniques like Kalman filters fuse encoder data with external sensors

for robust estimation.

> ***Example:*** *In a 100 m straight-line traversal, 1% slippage might cause a 1 m error; fusing with GPS reduces this to centimeters.*

Kinodynamic Planning

Wheeled robots' non-holonomic constraints require planning that respects kinematic limits on velocity and acceleration, leading to **kinodynamic planning**. This extends standard path planning by considering dynamic feasibility.

Sampling-Based Methods

These operate in the task space X and include:

- Probabilistic Roadmap (PRM).
- Rapidly-exploring Random Tree (RRT).

To handle non-holonomy, grow the search tree using kinematic primitives—predefined motion segments such as:

1. Straight line.
2. Forward left arc.
3. Forward right arc.

This ensures generated paths are executable without violating constraints.

Probabilistic Motion Primitives (ProMP)

ProMPs offer a probabilistic approach to trajectory generation ("Probabilistic Motion Primitives based Trajectory Planning", RSS 2021).

- Trajectories are represented as $\tau = \Phi w$, where $w \sim \mathcal{N}(\mu, \Sigma)$ is a weight vector drawn from a Gaussian.
- Primitives are generated via forward kinematics:
  1. Discretize the input space (e.g., velocity profiles).
  2. Record resulting trajectories from simulations.
  3. Compute mean μ and covariance Σ from the data.

Advantages over alternatives:

- Compared to lattice methods: Avoids rigid discretization of the space.
- Compared to optimization: Handles cluttered environments in real-time by sampling.

> ***Source:*** *https://www.roboticsproceedings.org/rss17/p058.pdf*; ***Video:*** *https://youtu.be/-CT4bpQUg?si=NFF5c1QOIyfGN9Lp*.

For implementation, a basic Python example for sampling a ProMP:

```
import numpy as np

# Define basis matrix Phi (e.g., for 10 time steps, 3 DoF trajectory)
Phi = np.random.rand(30, 5)  # 10 steps * 3 DoF x weights

# Mean and covariance for weights
mu = np.zeros(5)
Sigma = np.eye(5) * 0.1

# Sample weights
w = np.random.multivariate_normal(mu, Sigma)

# Generate trajectory
tau = Phi @ w
print("Sampled trajectory:", tau[:3])  # First 3 DoF at t=0
```

Dynamics

Dynamics extend kinematics by incorporating masses, inertias, and forces, essential for realistic simulation and control under acceleration.

Lagrange Method

The equations of motion are derived using the Lagrangian formulation:

> ***Lagrangian Dynamics***
>
> $$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}}\right) - \frac{\partial L}{\partial q} = A(q)^T \lambda + \tau$$
>
> *where L is the Lagrangian ($L = T - V$, kinetic minus potential energy), $A(q)$ enforces no-slipping constraints, λ are Lagrange multipliers for constraints, and τ are generalized forces/torques.*

This method systematically handles both holonomic and non-holonomic constraints.

Example: Differential Drive Robot

Consider a differential drive with these assumptions:

- Total mass m distributed along the body x_b-axis, at distance d from the axle A.
- Massless wheels for simplicity.
- No slip, no skid.

Preliminaries:

- State: $q = [x, y, \theta, \phi_L, \phi_R]^T$.
- Mass center: $x_G = x - d\sin\theta$, $y_G = y + d\cos\theta$.
- Velocities: $\dot{x}_G = \dot{x} - d\cos\theta\dot{\theta}$, $\dot{y}_G = \dot{y} - d\sin\theta\dot{\theta}$.
- Torques: $\tau = [0, 0, 0, \tau_L, \tau_R]^T$.
- $A(q)$ from no-slipping constraints.

The Lagrangian is $L = T - V$, with potential $V = 0$ (flat ground) and kinetic energy $T = \frac{1}{2}m(\dot{x}_G^2 + \dot{y}_G^2) + \frac{1}{2}\dot{\theta}^2 J(\theta)$, where $J(\theta)$ is the inertia tensor for rotation.

Applying Lagrange yields the dynamic equations:

> ***Dynamic Equations***
>
> $$M(q)\ddot{q} + C(q, \dot{q})\dot{q} = A(q)^T\lambda + \tau$$
>
> *Here, $M(q)$ is the mass/inertia matrix, and $C(q, \dot{q})$ captures Coriolis and centrifugal terms.*

Table of Key Matrices (Simplified for Constant Mass):

Component	Description	Form
$M(q)$	Inertia matrix	Diagonal with m for translations, J for rotation; wheel inertias added if non-zero
$C(q, \dot{q})$	Coriolis matrix	Terms like $-md\dot{\theta}^2\sin\theta$ for coupling
$A(q)^T\lambda$	Constraint forces	Enforces no-slip via multipliers λ

Simple vs. Complex Models

Simple models suffice for basic navigation, but complex ones are needed for high-speed or rough-terrain applications like autonomous driving or racing:

- Include massive wheels with their own inertias.

- Model actuators (e.g., motor dynamics).
- Incorporate tire-floor interactions (e.g., Pacejka magic formula for friction).
- Add suspension effects for load distribution.
- Account for aerodynamics at high speeds.

> ***Open-Source Simulators***
>
> - ***CARLA**: https://carla.org/ – A simulator for complex vehicle dynamics, including traffic and weather, ideal for testing car-like robots.*

Extension: Aerial Robots (Quadrotor)

To contrast with wheeled robots, we briefly cover aerial robots, which lack ground constraints but face underactuation challenges.

Types of Aerial Robots

Aerial robots vary, but quadrotors are common due to their simplicity and agility.

Quadrotor Model

The configuration has 6 variables (position and orientation in 3D), but it is underactuated with only 4 inputs (propeller speeds). Control is typically over position (3 DoF) and yaw angle (1 DoF), while roll and pitch are used internally for attitude.

Working Principle

Four propellers generate thrust and torque: clockwise/counter-clockwise pairs counter rotation, and differential speeds control tilt for direction.

Frames and State

- Body angular velocity: $\omega = [p, q, r]^T$ (roll, pitch, yaw rates).
- Relation to Euler rates:

$$\omega = \begin{bmatrix} 1 & 0 & -\sin\phi \\ 0 & \cos\phi & \sin\phi\cos\theta \\ 0 & -\sin\phi & \cos\phi\cos\theta \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

Dynamics (Newton-Euler)

For a rigid body with mass at the origin and low speeds (neglecting gyroscopics):

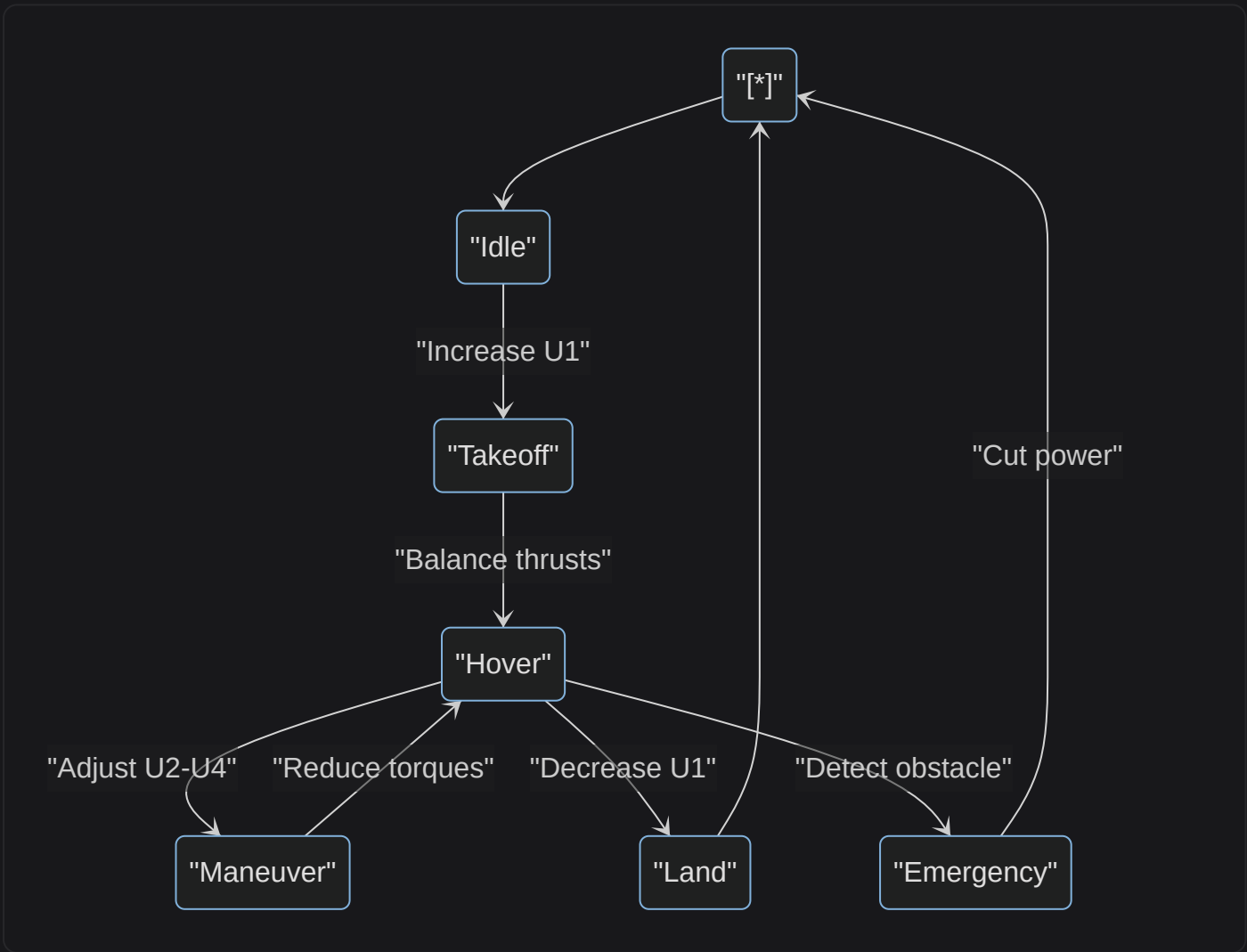
- Translational: $m\ddot{\xi} = Rf - mge_3$, where $f = [0, 0, \sum f_i]^T$ is total thrust, R is the rotation matrix from body to world, and $e_3 = [0, 0, 1]^T$.

- Rotational: $I\dot{\omega} = -\omega \times (I\omega) + \tau$, where $\tau = [\sum(-1)^i l f_i, \sum(-1)^{i+1} l f_i, \sum(-1)^i Q_i]^T$ (roll, pitch, yaw torques); l is arm length.
- Propeller forces: $f_i = k\omega_i^2$, torques $Q_i = b\omega_i^2$, with constants k, b.
- Collective inputs: $U_1 = \sum f_i$ (thrust); U_2, U_3, U_4 (roll, pitch, yaw torques).

> ***Example:*** *For hover, set $U_1 = mg$, $U_2 = U_3 = U_4 = 0$, yielding $\ddot{\xi} = 0$ and steady $\omega = 0$.*

> ***Open-Source Simulators***
>
> - ***Flightmare***: *https://uzh-rpg.io/flightmare/ – Focuses on photorealistic simulation for quadrotor swarms.*
> - ***AirSim***: *https://microsoft.github.io/AirSim – Integrates with Unreal Engine for realistic aerial dynamics and sensor simulation.*

To highlight the difference in planning, wheeled robots use ground-constrained paths, while quadrotors enable 3D free flight but require thrust limits—consider this state diagram for quadrotor modes:



References

- Unicycle Model
- Differential Drive
- Car-Like Robots
- Non-Holonomic Constraints
- Quadrotor Dynamics